

HOVERCRAFT: A SCALABLE RAFT MODIFICATION – TLA+ DESIGN AND LIVENESS CONSIDERATIONS

Presented by Parsa VARES – 16.06.2025

Dr. Ovidiu-Cristian MARCU & Dr. Georgios KAFANAS

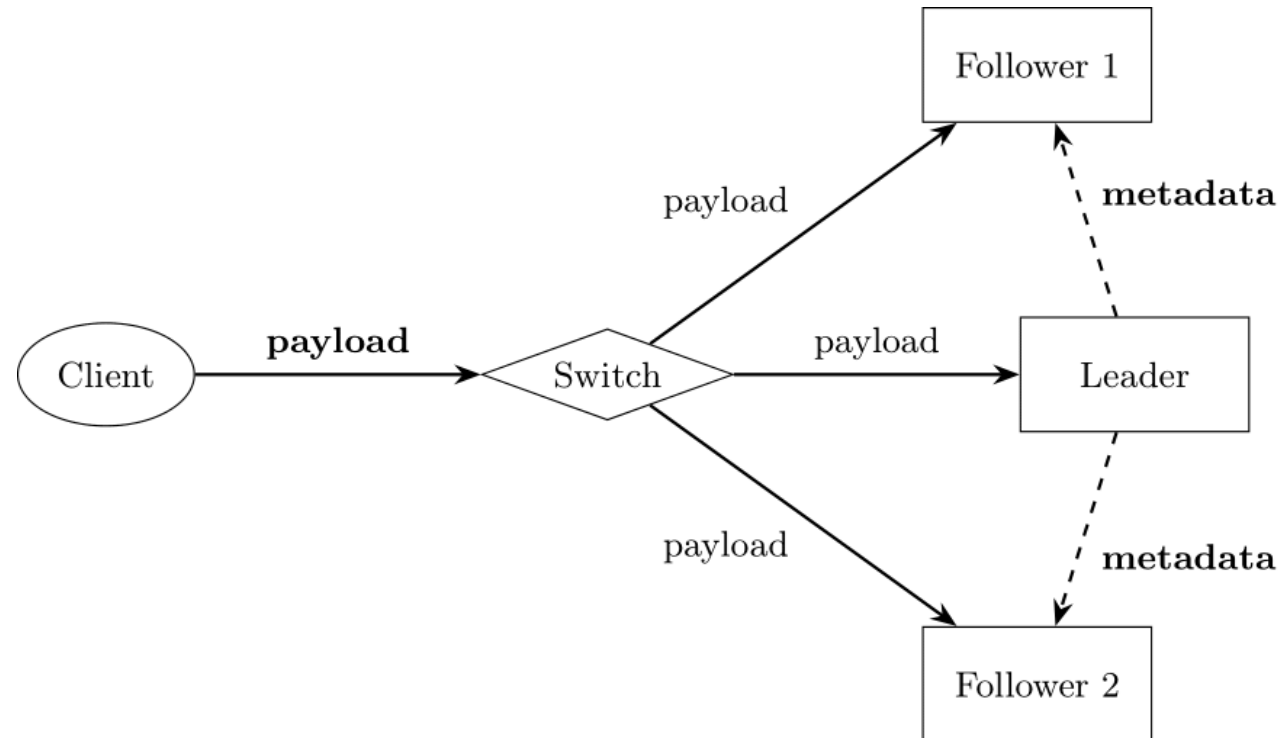




CONTENT:

- ❖ Hovercraft Design – The Switch & Payload Delivery
- ❖ Hovercraft Design – Leader Ordering & Follower Reconstruction
- ❖ Liveness in Hovercraft – Fundamentals & Key Stages
- ❖ Liveness: Replication, Reconstruction & Recovery
- ❖ Overall Liveness Discussion & Project Conclusion

❖ HOVERCRAFT CORE DESIGN: DECOUPLING PAYLOAD & ORDER



Key idea: Separate large payload dissemination from consensus ordering

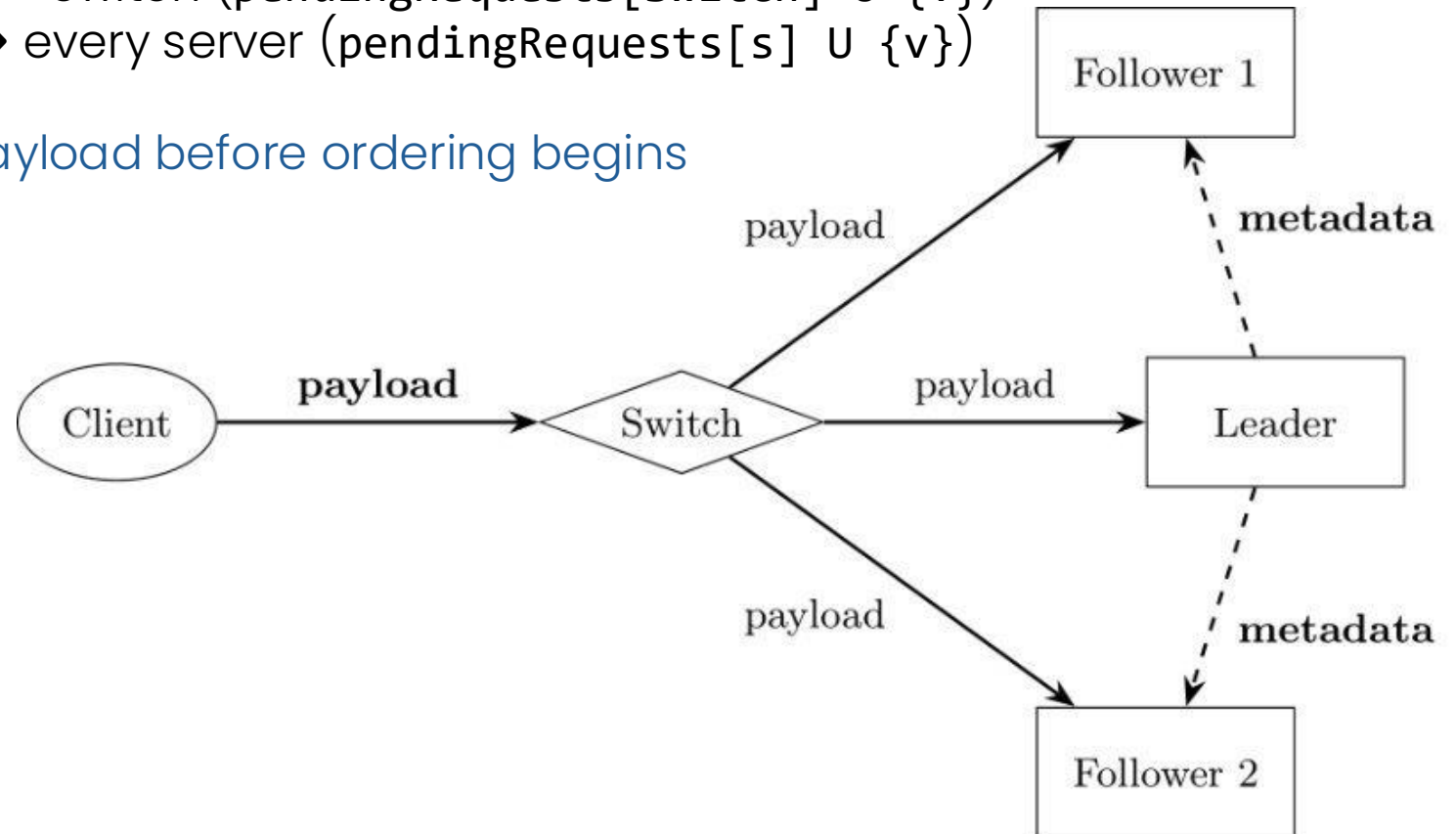
1. Switch component

Receives client payloads once and Multicasts payloads to *all* Raft servers

2. TLA+ variables / actions

- **CONSTANTS** Switch – distinct entity
- `pendingRequests[node]` – per-node payload buffer
- `SwitchClientRequest(v)` – client $\rightarrow$ Switch (`pendingRequests[Switch] U {v}`)
- `SwitchDisseminate(v)` – Switch $\rightarrow$ every server (`pendingRequests[s] U {v}`)

Outcome: All servers possess the raw payload before ordering begins



❖ HOVERCRAFT DESIGN – LEADER ORDERING & FOLLOWER RECONSTRUCTION

3. LeaderOrderRequest(i, v)
 - Leader selects v from pendingRequests[i]
 - Appends full entry [term, value=v, payload=v] to log[i]
4. AppendEntries(i, j)
 - Sends *metadata only* [term, value] to follower j
5. HandleAppendEntriesRequest(i, j, m) (*follower side*)
 - Looks up v in pendingRequests[i]
 - Reconstructs full entry; appends to log[i]
 - If payload missing → initiates *RecoveryRequest*

Benefit: Leader's replication path carries minimal data; payload never resent

❖ LIVENESS IN HOVERCRAFT – FUNDAMENTALS & KEY STAGES

Global Liveness Theorem

$$\Diamond \exists i \in \text{Server}, idx : \log_i[idx].value = v \wedge idx \leq commitIndex_i$$

Under *weak fairness* of enabled actions and *eventual message delivery*.

What is Liveness?

- “Something good eventually happens.”
- Client requests are eventually processed and committed.
- Depends on a stable leader (standard Raft liveness).

Key Hovercraft Stages

1. Payload Dissemination (Switch → Servers)
 - Will every non-faulty server’s pendingRequests cache receive the payload?
 - Requires SwitchClientRequest & SwitchDisseminate actions to eventually fire (fair Switch scheduling).
2. Leader Ordering (Leader Log)
 - Will a payload in the leader’s pendingRequests be ordered and logged?
 - Requires LeaderOrderRequest to be fairly scheduled under a stable leader.

❖ LIVENESS – REPLICATION, RECONSTRUCTION & RECOVERY

```
EventuallyClientRequestProcessed(v) ==  
  <> ∃ i ∈ Server, idx ∈ DOMAIN log[i] :  
    log[i][idx].value = v ∧ commitIndex[i] ≥ idx  
  
EventuallyLeaderOrdersPendingRequest(leader, v) ==  
  (v ∈ pendingRequests[leader]) ~>  
  <> ∃ idx ∈ DOMAIN log[leader] : log[leader][idx].value = v
```

Continued Stages:

3. Metadata Replication (Leader → Followers)

- Will [term, value-ID] references reach a quorum via AppendEntries?
- Relies on Raft's retry logic and eventual message delivery.

4. Follower Log Construction

- With payload in cache + metadata from leader, will follower reconstruct and append [term, value, payload]?
- Requires HandleAppendEntriesRequest to run fairly.

5. Payload Recovery (*loss-mitigation*)

- If initial multicast is missed, will RecoveryRequest/Response eventually deliver the payload?
- Depends on leader availability and reliable exchange of recovery messages.

❖ LIVENESS DISCUSSION & CONCLUSION

Liveness Dependencies:

Standard Raft: leader stability, reliable server-to-server links, live processes.

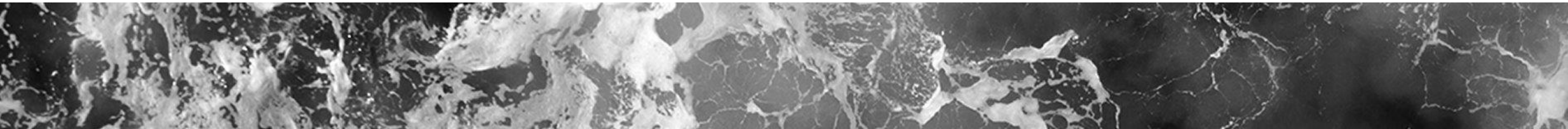
Hovercraft Additions:

- Switch liveness (process SwitchClientRequest / SwitchDisseminate).
- Reliable Switch → Server payload delivery (mitigated by recovery loop).

Resource bounds: pendingRequests limited by MaxClientRequests.

Hovercraft++ Impacts

- Batching metadata boosts throughput; does not alter core liveness assumptions.
- Leaderless/optimistic reads follow a distinct progress path.



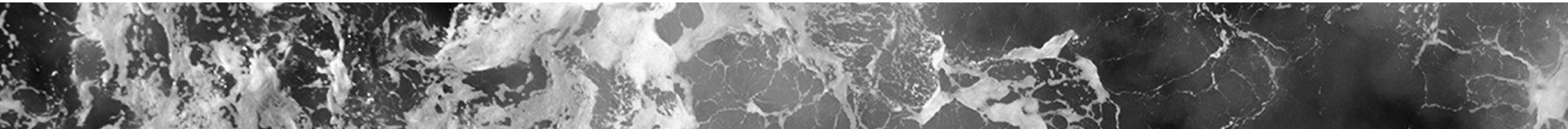
Conclusion:

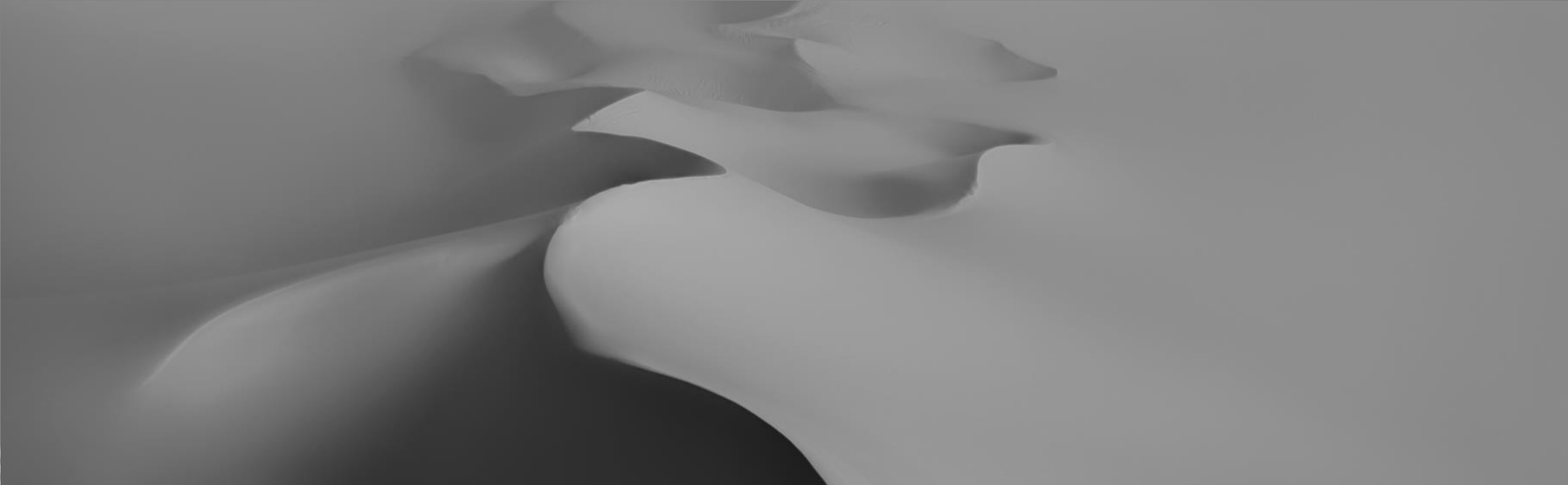
Safety: TLA+ invariants (LogInv, LeaderCompletenessInv, ...) upheld; payload–metadata separation preserves Raft guarantees.

Liveness: End-to-end progress hinges on:

1. Switch disseminates payload.
2. Stable leader orders entry.
3. Metadata replicates to quorum.
4. Followers reconstruct entry (or recover payload).

Key Takeaway: Decoupling large payloads from ordering improves scalability while retaining Raft-style safety and progress—provided the Switch and recovery mechanism remain live.



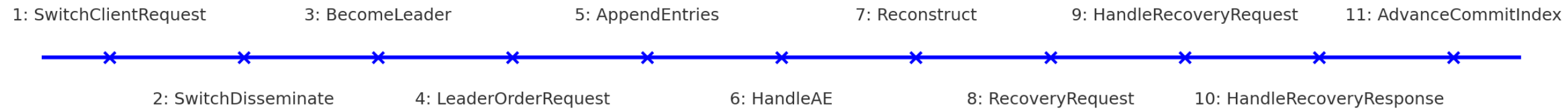


THANK YOU

Presented by Parsa VARES
Repository: [GIT](#)

❖ Appendix: Formal Liveness Timeline

Hovercraft Liveness Timeline (11 Steps)



Liveness Timeline for Client Request v

Step 1: Client $\rightarrow$ Switch SwitchClientRequest(v)

Precondition: $v \notin \text{pendingRequests}[\text{Switch}]$

Postcondition: $v \in \text{pendingRequests}'[\text{Switch}]$

Temporal Property:

$$\Diamond (v \in \text{pendingRequests}[\text{Switch}])$$

Step 2: Switch $\rightarrow$ Servers SwitchDisseminate(v)

Precondition: $v \in \text{pendingRequests}[\text{Switch}]$

Postcondition:

$$\forall s \in \text{Server} : v \in \text{pendingRequests}'[s] \wedge v \notin \text{pendingRequests}'[\text{Switch}]$$

Temporal Property:

$$(v \in \text{pendingRequests}[\text{Switch}]) \rightsquigarrow \Diamond \forall s. v \in \text{pendingRequests}[s]$$

Step 3: Leader Election BecomeLeader(L)

Precondition:

$$\text{votesGranted}[L] \in \text{Quorum} \wedge \text{leaderCount}[L] < \text{MaxBecomeLeader}$$

Postcondition:

$$\text{state}'[L] = \text{Leader} \wedge \text{currentTerm}'[L] = T$$

Temporal Property:

$$\Diamond \exists L. (\text{state}[L] = \text{Leader})$$

Step 4: Leader orders LeaderOrderRequest(L, v)

Precondition:

$$\text{state}[L] = \text{Leader} \wedge v \in \text{pendingRequests}[L]$$

Postcondition:

$$\log'[L] = \text{Append}(\log[L], [\text{term}, \text{value} = v, \text{payload} = v]) \wedge v \notin \text{pendingRequests}'[L]$$

Temporal Property:

$$(\text{state}[L] = \text{Leader} \wedge v \in \text{pendingRequests}[L]) \rightsquigarrow \Diamond \exists idx. \log[L][idx].\text{value} = v$$

Step 5: Send metadata AppendEntries(L, F)

Precondition:

$$\text{nextIndex}[L][F] \leq \text{Len}(\log[L])$$

Postcondition:

$$\text{messages}' = \text{messages} \cup \{m \mid m.\text{mentries} = \langle \langle \text{term}, \text{value} = v \rangle \rangle\}$$

Temporal Property:

$$(\exists idx. \log[L][idx].\text{value} = v) \rightsquigarrow \Diamond \exists m. (m \in \text{messages} \wedge m.\text{dest} = F \wedge m.\text{mentries}[1].\text{value} = v)$$

Step 6: AE arrival HandleAppendEntriesRequest(F, L, m)

Precondition:

$$m \in \text{messages} \wedge m.\text{mtype} = \text{AppendEntriesRequest} \wedge m.\text{mentries} \neq \langle \rangle$$

Postcondition: branches 7 and 8 handle payload present/missing

Temporal Property:

$$\Diamond (m.\text{mentries} \neq \langle \rangle \wedge m \in \text{messages}) \rightsquigarrow \Diamond \text{HandleAppendEntriesRequest}(F, L, m)$$

Step 7: Reconstruct (present)

HandleAppendEntriesRequest branch when payload present

Precondition:

$$v \in \text{pendingRequests}[F] \wedge m.\text{mentries}[1].\text{value} = v$$

Postcondition:

$$\log'[F] = \text{Append}(\log[F], \text{fullEntry}) \wedge v \notin \text{pendingRequests}'[F]$$

Temporal Property:

$$(v \in \text{pendingRequests}[F] \wedge \text{AE delivered}) \rightsquigarrow \Diamond \exists i. \log[F][i].\text{payload} = v$$

Step 8: RecoveryRequest (missing)

HandleAppendEntriesRequest branch when payload missing

Precondition:

$$v \notin \text{pendingRequests}[F] \wedge m.\text{mentries}[1].\text{value} = v$$

Postcondition:

$$v \in \text{missingRequests}'[F] \wedge \text{RecoveryRequest}(v) \in \text{messages}$$

Temporal Property:

$$(v \notin \text{pendingRequests}[F] \wedge \text{AE delivered}) \rightsquigarrow \Diamond (v \in \text{missingRequests}[F] \wedge \text{RecoveryRequest sent})$$

Step 9: Leader RecoveryResponse HandleRecoveryRequest(L, F, m)

Precondition:

$$m.\text{mtype} = \text{RecoveryRequest} \wedge v \in \text{missingRequests}[F] \wedge v \in \log[L]$$

Postcondition:

$$\text{RecoveryResponse}(v) \in \text{messages}$$

Temporal Property:

$$(v \in \text{missingRequests}[F]) \rightsquigarrow \Diamond \exists m. (m.\text{mtype} = \text{RecoveryResponse} \wedge m.mRequestValue = v)$$

Step 10: Follower Recovery HandleRecoveryResponse(F, L, m)

Precondition:

$$m.\text{mtype} = \text{RecoveryResponse} \wedge m.mRequestValue = v$$

Postcondition:

$$v \in \text{pendingRequests}'[F] \wedge v \notin \text{missingRequests}'[F]$$

Temporal Property:

$$(\exists m. m.\text{mtype} = \text{RecoveryResponse} \wedge m.mRequestValue = v) \rightsquigarrow \Diamond (v \in \text{pendingRequests}[F])$$